

Reviewer Pack — Minimal Ehrhart Quotient Correlation with Frege Proof Depth

Entry ea928ad95433 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 15:52:02 UTC

Minimal Ehrhart Quotient Correlation with Frege Proof Depth

Entry ID: ea928ad95433 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-01 15:52:02 UTC

1. Conjecture

Field A (mathematical branch): Ehrhart Theory **Field B** (complexity object): Frege Proof Complexity

Statement:

For every Boolean function φ represented by a 2CNF, the minimal Ehrhart quotient ($q(\varphi)$) of its associated variety is linearly correlated with its Frege proof depth $d(\varphi)$, such that $q(\varphi) = \Theta(d(\varphi))$.

Rationale (proposer's reasoning):

Ehrhart Theory provides a combinatorial approach to counting integer points in polytopes, which can be applied to the evaluation of computational problems. The Ehrhart quotient measures the complexity of the variety associated with the Boolean function and may reveal structural properties related to proof complexity.

Taxonomy category: Ehrhart_Theory_to_Frege_Proof (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): dcbebccd62811c78

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for at least 80% of the tested 2CNFs with n variables ($n \leq 40$), the linear regression coefficient of $q(\varphi)$ vs. $d(\varphi)$ exceeds 0.7, and the p-value is less than 0.05.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Ehrhart Theory" AND "Frege Proof Complexity" AND minimal Ehrhart quotient - "2CNF variety" AND linear correlation AND Ehrhart quotient Frege proof depth - Ehrhart Theory intersection Frege proof complexity CNF minimal quotient

Top relevant hits considered: - [http://arxiv.org/abs/0904.3035v2] Additive number theory and inequalities in Ehrhart theory - [http://arxiv.org/abs/2402.10351v3] Quantum Automating $\mathbf{TC}^0$ -Frege Is LWE-Hard - [http://arxiv.org/abs/2211.17129v2] Ehrhart Limits

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=0.4s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_2cnf(n):
        clauses = []
        for _ in range(2 * n):
            literals = [random.choice([1, -1]) * (i + 1) for i in range(n)]
            clause = random.sample(literals, k=2)
            clauses.append(clause)
        return clauses

    def ehrhart_quotient(n):
        # Placeholder function to compute the Ehrhart quotient
        # This is a dummy implementation and should be replaced with actual computation
        return n + 1

    def frege_proof_depth( $\phi$ ):
        # Placeholder function to compute the Frege proof depth
        # This is a dummy implementation and should be replaced with actual computation
        return len( $\phi$ )

    results = []
    for n in [5, 10, 15, 20, 30, 40]:
        for _ in range(5):
             $\phi$  = generate_2cnf(n)
            q_n = ehrhart_quotient(n)
            d_n = frege_proof_depth( $\phi$ )
            results.append((q_n, d_n))
```

```

if not results:
    return {
        "metric_name": "correlation_coefficient",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

n = len(results)
sum_q_n = sum(q_n for q_n, _ in results)
sum_d_n = sum(d_n for _, d_n in results)
sum_q_n_squared = sum(q_n**2 for q_n, _ in results)
sum_d_n_squared = sum(d_n**2 for _, d_n in results)
sum_q_n_d_n = sum(q_n * d_n for q_n, d_n in results)

mean_q_n = sum_q_n / n
mean_d_n = sum_d_n / n

covariance = sum_q_n_d_n - n * mean_q_n * mean_d_n
variance_q_n = sum_q_n_squared - n * mean_q_n**2
variance_d_n = sum_d_n_squared - n * mean_d_n**2

if variance_q_n == 0 or variance_d_n == 0:
    return {
        "metric_name": "correlation_coefficient",
        "metric_value": None,
        "instances_tested": n,
        "n_max": max(n for _, n in results),
        "conjecture_holds": False,
        "counterexample": "variance_zero"
    }

correlation_coefficient = covariance / (math.sqrt(variance_q_n) * math.sqrt(variance_d_n))

return {
    "metric_name": "correlation_coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": n,
    "n_max": max(n for _, n in results),
    "conjecture_holds": correlation_coefficient > 0.7,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

```


11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15143
2	preregistration	ollama_remote	glm4:latest	0	0	9450
3	novelty	ollama_remote	glm4:latest	0	0	8636
4	novelty	ollama_remote	glm4:latest	0	0	18701
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14089
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20386
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10576
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18922
9	critic	ollama_remote	glm4:latest	0	0	10965
10	judge	ollama_remote	glm4:latest	0	0	9389

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 136258 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/ea928ad95433.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/ea928ad95433.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/ea928ad95433.tar.gz` (if generated)